

The F1R3Skein Virtual Instrument

Specification, revision 4

F1R3FLY.io

4 September 2026

Abstract

This document specifies the F1R3Skein virtual instrument: the gestural surface through which a musician draws melody out of the digit streams of mathematical constants, and out of which the platform’s two primary artifacts—tunes and performances—are produced.

Revision 3 replaced the instrument’s physical model. Revision 4 settles the ten decisions that model raised and adds the organising principle that had been missing:

While M holds the ribbons she is playing. While the ribbons are mounted she is in meta play.

That principle removes machinery rather than adding it. What revision 3 specified as a suppression rule—“pull must be inhibited during a cut”—is not a rule at all: pull does not exist in meta play because there are no ribbons in her hands to pull. The recogniser evaluates one mode’s predicates at a time.

Also settled here: the cut is *virtual*, so the band is never severed; the zip wave has a bounded frontier and the far field compresses so that long plays visibly cost depth; tempo is read from the speed at which M closes her hands and is fixed for the life of a mesh; and halt is a head gesture, which removes the eyes from the vocabulary entirely and with them the visionOS gaze constraint that dominated revision 3.

The vocabulary is ten gestures, each with a diagram. Section 18 lists what remains open, and §17 triages the set against a demo deadline.

Contents

1	Scope and conformance	2
1.1	Terminology	3
2	Deployment configurations	3
3	The generative core	3
3.1	Streams	3
3.2	Two cursors, separately mobile	4
3.3	Note mapping	4
4	The playing surface	4
5	Instrument state	5
5.1	The two modes	5
6	Gesture vocabulary	5
6.1	Pull (play)	6
6.2	Twist (play)	6
6.3	Zip (play)	6

6.4	Halt (play)	7
6.5	Unzip (play)	8
6.6	Mount (play $\rightarrow$ meta)	8
6.7	Unmount (meta $\rightarrow$ play)	9
6.8	Loop (meta)	9
6.9	Snip (meta)	10
6.10	Mutual exclusion	11
6.11	Three kinds of detector	11
6.12	Detection parameters	11
6.13	Finger curl must be scale-invariant	12
7	Snip semantics and the tune term	12
7.1	The defect being corrected	12
7.2	Resolution	12
7.3	Consequences	13
7.4	Interpretation lives in the envelope	13
8	Performance recording	13
9	Wire protocol, version 3	14
9.1	Client to engine	14
9.2	Engine to client	14
9.3	Rate discipline	15
10	Visual model	15
11	Audio model	15
12	Non-functional requirements	16
13	Instrumentation	16
14	Publishing and consent	16
15	Deferred	17
16	Conformance checklist	17
17	Triage against the demo deadline	18
18	Decisions sought	18

1 Scope and conformance

In scope: the generative core, the playing surface and its geometry, the instrument’s state model, the gesture vocabulary and its detection criteria, the wire protocol, the visual and audio models, performance recording, and the budgets that make the instrument playable.

Out of scope: identity, galleries, the engagement ledger, breeding and selection, and the on-chain representation of tunes. This document defines what the instrument *produces*. The instrument is to be brought up and proven standalone; wiring it to a shard follows, and nothing here MAY be made to depend on shard availability.

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are used in the sense of RFC 2119. *M* denotes the musician throughout.

1.1 Terminology

<i>Engine</i>	The process holding authoritative instrument state.
<i>Client</i>	The visionOS application: gesture recognition, rendering, audio.
<i>Stream</i>	One spigot: a constant, an output base, a cursor position.
<i>Spool</i>	The visible source of one stream, far from M on the z -axis.
<i>Ribbon</i>	The visible material spooling from one spool toward M.
<i>Notch</i>	One digit position, presented on a ribbon’s inner edge.
<i>Skein</i>	The pair of ribbons together with their relative registration.
<i>Zip front</i>	The boundary where unmeshed ribbon becomes meshed. The playhead.
<i>Tune</i>	A snapped, named artifact.
<i>Performance</i>	The recorded trace of a session.

2 Deployment configurations

S1 — Tethered.

Engine on a Mac, client on the headset, TCP over the local network with Bonjour discovery. September test configuration; explicitly a stopgap.

S2 — On-device.

Engine built for `arm64-apple-visionos` as a static library behind a C foreign-function interface. No transport at all. The target.

The protocol of Section 9 is defined over an abstract bidirectional channel; in S1 that is a TCP stream, in S2 a pair of FFI ring buffers. No other part of this specification distinguishes them, and the client **MUST NOT** contain transport-specific logic outside its channel implementation.

The Unix-domain-socket transport **MUST NOT** be used for device testing: a Unix socket is addressed by a path in one host’s filesystem, and the headset is a different host.

3 The generative core

3.1 Streams

The engine **MUST** provide the five constants— π , e , φ , $\ln 2$, Liouville’s—each emitting digits in a configurable base b , $2 \leq b \leq 36$. Emission **MUST** be exact and unbounded. A stream configuration is a pair (c, b) ; with a cursor position p it determines the digit $d(c, b, p)$ uniquely.

The π implementation **SHOULD** move to Bailey–Borwein–Plouffe extraction where the base permits, and **MUST** memoise emitted digits so re-realisation of a visited range is $O(1)$ per digit. Deep positions are exactly what a long session produces.

3.2 Two cursors, separately mobile

The skein holds two configurations and two *independent* cursor positions, p_L and p_R . Independence is the point of the design and MUST be preserved: it is what lets M hold a rhythmic motif fixed while advancing pitch, or dually.

Proposition 1 (The instrument performs its own algebra). *Holding one ribbon fixed and meshing a different segment of the other against it takes the durations of one candidate and the pitches of another. That is the crossover operator XPitch of the breeding engine, performed by hand; its dual is XPitch(f, m) by the rectangular-band identity. Selection and reproduction are not a subsystem bolted onto a music toy—the population does at scale what M does at the bench.*

3.3 Note mapping

One stream governs pitch, the other duration; *twist* exchanges the roles. For the k -th meshed notch pair the engine MUST produce $(\Pi(d_{\text{pitch}}), \Delta(d_{\text{duration}}))$. Both maps MUST be session parameters, not build constants. Pitch maps MUST include major, minor, pentatonic major, pentatonic minor, Dorian, whole tone and chromatic over a configurable root; duration maps MUST include musical, linear, exponential and fixed. A stream’s base SHOULD match the cardinality of the map it drives.

Both maps MUST be changeable during a session. For the first release the control is **textual**, in the panel, and MUST NOT consume a gesture: the vocabulary is already at ten, and choosing a scale is a settings act rather than a performing one. A gestural treatment may follow. Because a changed map changes how an existing term sounds, the capture envelope of §7.4 carries the maps in force at the moment of capture.

4 The playing surface

Axes are M’s: x is left–right, y is up–down, z is depth away from M.

Each stream has its own spool. The two spools sit at the *same height* and are spread apart on the x -axis, far from M on z . Ribbon spools from each toward M’s hands. Each ribbon divides into three sections:

Future still wound on the spool, unseen.

Present

between the spool and M’s hands, visible.

Past having passed the hands, hanging below them toward the floor.

Each ribbon carries notches on its *inner* edge—the edge facing the other ribbon. Bringing the ribbons together meshes the notches over a section of the present, leaving unmeshed margins near the hands and near the spools. The mount sits between the ribbons, just in front of the hands, and is a pair of clips.

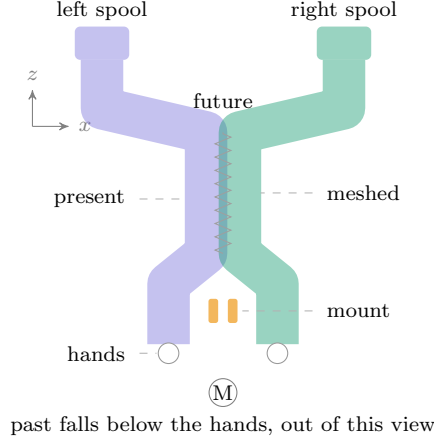


Figure 1: The playing surface, seen from above. Two spools at the same height, spread on x .

A notch pair is a note. Meshing commits notch k on the left to notch k on the right. While unmeshed the ribbons slide independently and M sets the relative phase; once meshed the pairing is fixed. Independent mobility and well-formed pairing are not in tension—they are separated in time by the zip.

5 Instrument state

$$\Sigma = \langle (c_L, b_L, p_L), (c_R, b_R, p_R), m, \zeta, \lambda, \Pi, \Delta, \tau, \iota, T \rangle$$

with $m \in \{\text{play}, \text{meta}\}$ the mode, ζ the zip state (unzipped, or zipped with a front position, a propagation state and a frontier budget), λ the loop state, τ the tempo of the current mesh, ι the instrument, T the tray. The client MUST NOT hold authoritative state; it renders Σ and emits gestures.

5.1 The two modes

	Play	Meta
Ribbons	in M 's hands	in the mount clips
Wave	propagating, or halted	stopped
Gestures	pull, twist, zip, unzip, halt	loop, scissors
Transition out	mount	unmount

The recogniser MUST evaluate only the active mode's predicates. This is not an optimisation: it is what makes the vocabulary safe. Travelling a hand along z to bracket a cut is indistinguishable from a pull, and the two never collide only because pull does not exist in meta play.

Mounting while unzipped is permitted and yields a meta play in which loop and scissors are both unavailable, there being nothing meshed to loop or to cut.

6 Gesture vocabulary

Ten gestures, grouped by mode. Each is given with its physical form, its effect, and a diagram of the surface during it.

6.1 Pull (play)

M draws one hand toward herself, advancing that stream. Step count is proportional to speed. Pulls are per-hand and independent; this is where the relative phase between the two streams is set.

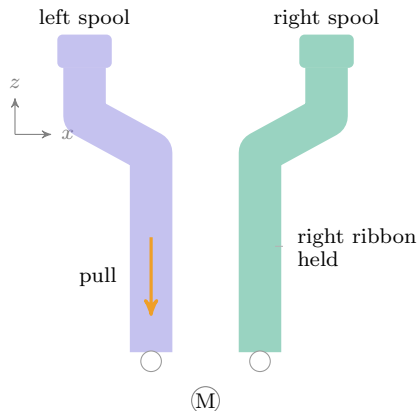


Figure 2: Pull. Ribbons unmeshed; each advances alone. Holding one and pulling the other is how M chooses which rhythm meets which pitches.

6.2 Twist (play)

One hand held clearly above the other, hands apart. Exchanges the two streams, cursors included. Twist MUST be permitted only while unzipped: the mesh commits the pairing, and exchanging roles underneath a committed mesh has no coherent reading.

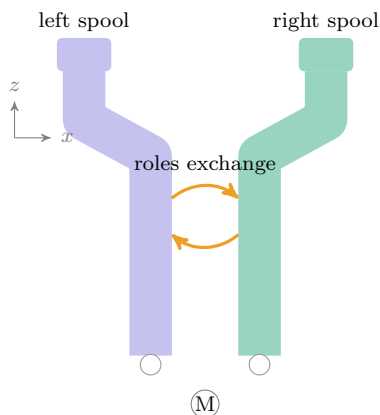


Figure 3: Twist. Which stream governs pitch and which governs duration is swapped. Permitted only while unzipped.

6.3 Zip (play)

M brings the two ribbons together. The notches mesh, and **the mesh keeps going**: the zip front propagates away from M into the future without further input. The zip front is the playhead — each notch pair sounds as it engages.

Tempo is read from the closing. The speed at which M's hands come together sets the propagation rate. The client MUST measure closing speed over a short window *before* the zip threshold is crossed, not at the crossing frame, or the tempo will be noisy.

One tempo per mesh. Halting and resuming involves no closing motion, so there is no new speed to read and the tempo carries over. A mesh therefore has a single tempo for its whole life, from the zip that created it until it is unzipped. This is a simplification worth keeping: it makes a captured region’s tempo unambiguous.

The frontier is bounded. The wave **MUST** stop of its own accord after `zip.frontier_max` notches. The bound is the length of a generous looper phrase — on the order of 10^3 notches at a few notes per second — and **SHOULD** be set from a reference device rather than guessed. Long before the bound is reached the spools **MUST** have compressed toward point sources, so that a long play visibly costs z -axis real estate.

There is an alignment here that is not contrived: a long play costs depth in the scene and costs phlo in the engine, because deep cursor positions are more expensive to realise. The metaphor and the economics say the same thing.

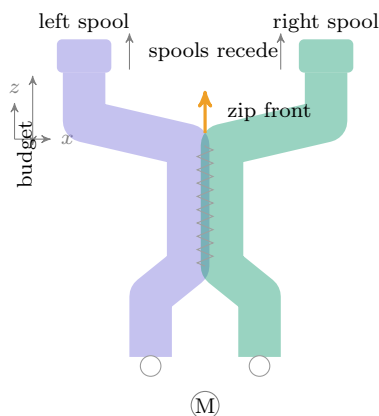


Figure 4: Zip. The wave propagates unaided; the spools retreat to feed it, up to a bounded frontier. The closing speed of the hands set the tempo.

6.4 Halt (play)

M tilts her head to stop the wave. The front freezes, the mesh is untouched, and the ribbons stay in her hands. **She has not stopped performing — she has stopped the generation of music.** Halt is how M places a silence longer than any rest the duration stream will hand her, and it is the only gesture in the vocabulary that composes rather than manages.

Why the head. Halt is the one transport control M needs while both hands are committed to holding ribbons. Head pose is available to applications — it is the channel analytics vendors fall back on precisely because gaze is not — and it is otherwise idle during play. The vocabulary then divides cleanly: hands move material, head handles transport. The authorization requirements of the device-pose provider **MUST** be confirmed before this is built.

Roll, not pitch or yaw. Yaw is in constant use, as M looks between the tray, the spools and the front. Pitch is what people do to keep time, and nodding along would fire it continuously. Roll is the least-used axis in ordinary playing — though not unused, since players lean into a phrase by reflex, so the threshold **MUST** be generous, **MUST** require a hold, and **MUST** live in the calibration profile.

Set and clear, not toggle. One direction halts, the other resumes; a repeat in the same direction is a no-op. A toggle on a noisy detector is unsafe: a missed fire and a double fire are indistinguishable to M, and she will correct by tilting again, which under a toggle undoes the

correction. The gesture **MUST** be edge-triggered with hysteresis, firing near 15° and re-arming below 6° , requiring a return to near-upright between firings. This matches the direction-sensitive treatment of loop, so the two feel consistent.

Confirmation and equivalence. A head tilt is proprioceptively silent — it does not feel like an action the way closing the hands does — so the freezing of the front **MUST** be given unmistakable visual confirmation. A panel equivalent **MUST** exist, since a head-only control is unavailable to anyone who cannot tilt comfortably.

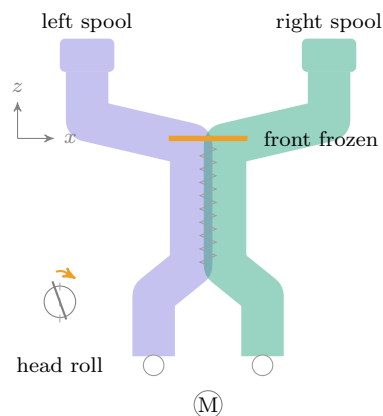


Figure 5: Halt. A head tilt freezes the front; the mesh and the tempo survive. The opposite tilt resumes, and the tempo carries over because there is no closing motion to read a new one from.

6.5 Unzip (play)

Separates the ribbons, retiring the mesh from the near end. Unzipping is what lets M re-set the relative phase and mesh a different segment against a held ribbon — the crossover move of Proposition 1. It is also the only way to change tempo, since a new mesh takes a new closing speed.

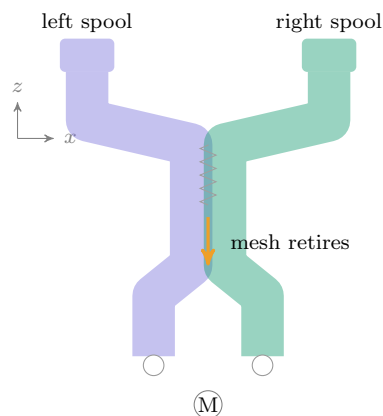


Figure 6: Unzip. The pairing is released from the near end, freeing the ribbons to slide independently again.

6.6 Mount (play $\rightarrow$ meta)

M slots the unmeshed lead of each ribbon into the clips. Mounting **stops the wave** and enters meta play. It is a clear gestural declaration that M is about to do something else with her

hands, and the mount takes over the holding so that both hands are free to travel out along the meshed band. The mount contributes no boundary to any captured tune.

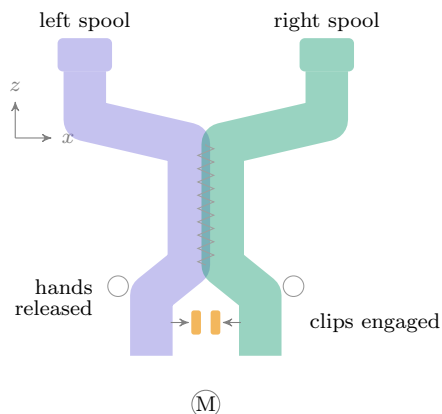


Figure 7: Mount. Two clips take the load, the wave stops, and M enters meta play.

6.7 Unmount (meta $\rightarrow$ play)

M reaches into the clips and takes the ribbons back. **The mesh survives.** She picks up a still-zipped assembly with the wave stopped, and resumes propagation from the current front — at the mesh’s existing tempo, by §6.3. The session rhythm is play, capture, play.

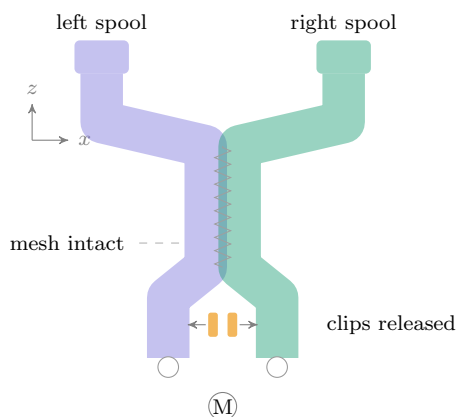


Figure 8: Unmount. The ribbons return to M’s hands with the mesh and the tempo preserved.

6.8 Loop (meta)

A circular finger motion — the “go on” gesture — sets the meshed section looping so M can audition it. The loop span runs from the mount to the stopped zip front. Reversing the direction of rotation clears the loop.

Looping needs no new machinery in the representation: repetition is **Seq** of copies, which the monoid already supplies. Loop is a transport control, recorded in the performance trace, and leaves the tune term untouched.

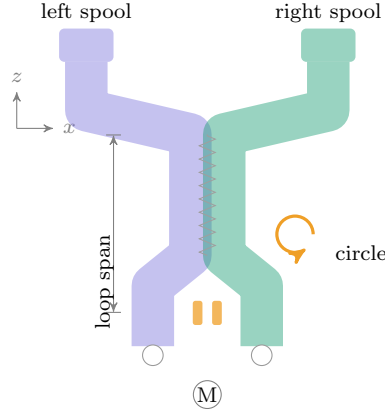


Figure 9: Loop. Mount to stopped front. Audition wide, cut narrow.

6.9 Snip (meta)

Both hands are involved. One hand is nearest M, the other nearest the spools; what is snipped is the section *between the hands*.

The cut is virtual. A *copy* of the bracketed region is lifted into the tray. The zipped band itself is undisturbed — nothing is severed, nothing falls into the past, and no stub is left clipped to the mount. Two consequences follow. M may take overlapping captures, the same region twice, or work back over material she has already harvested; successive snips from one run are therefore *not* required to be adjacent. And the visual **MUST** show the copy lifting clear while the band remains whole, or M will expect a gap that is not there.

- The near hand supplies i_L and i_R ; the separation of the hands in notches supplies n .
- Roles are assigned by z -ordering at the moment of the gesture, **MUST NOT** by chirality. Either hand may be the near one.
- Both boundaries **MUST** lie inside the meshed span. Bracketing unmeshed lead would capture two unpaired ribbons rather than a tune.
- The capture **MUST** be previewed: while the pose is held, a marker tracks each hand along the band so M can slide to the right notch before closing.

M is free to capture a smaller section than the full extent from mount to front. The three decisions are independent: the mount fixes where holding is taken over, the wave's stopping fixes how much material exists, the capture fixes how much she keeps.

Silence is not captured. A halt consumes no notches, so a capture spanning a halt yields consecutive notes and the silence is absent from the tune. The silence is preserved in the performance trace, which replays exactly. Carrying rests into captures is a deliberate deferral; see §15.

Naming. A capture **MUST** be auto-named at the moment it is taken, and **MUST** be renameable in the tray. M is in meta play with her hands free, so a prompt would interrupt nothing — but text entry on the headset is slow, and naming is a curatorial act rather than a performing one. The **scissors** message therefore carries no name.

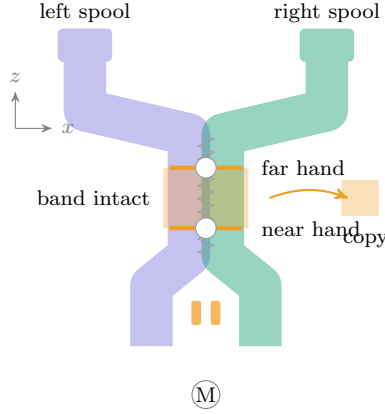


Figure 10: Snip. Two boundaries, both inside the mesh. A copy of what lies between the hands is lifted to the tray; the band itself is untouched.

6.10 Mutual exclusion

- The recogniser **MUST** evaluate only the active mode's predicates.
- Loop and pull **MUST NOT** be simultaneously satisfiable: a circling hand crosses the pull velocity threshold twice per revolution. Under the mode rule they never coexist, but the exclusion **MUST** be explicit in case modes are ever relaxed.
- Twist requires unzipped; zip and twist are never simultaneously satisfiable.
- Every gesture **MUST** be edge-triggered with a cooldown and **MUST NOT** re-fire while its pose is continuously held.

6.11 Three kinds of detector

The recogniser draws on three input sources and **MUST** support three detector kinds. Only the first exists today.

Kind	Source	Gestures
Per-frame pose	hand anchors	pull, twist, zip, unzip, mount, unmount, scissors
Trajectory	hand anchors	loop
Device pose	head anchor	halt

The trajectory detector **MUST** buffer index-fingertip positions over a window, fit a plane, accumulate the unwrapped angle about the centroid, and fire when the sweep passes 2π with radius variance and plane residual inside bounds. Direction of rotation comes free and gives loop-set against loop-clear.

6.12 Detection parameters

Every threshold **MUST** live in an external calibration profile, loaded at launch and reloadable without a rebuild. Hardcoded constants **MUST NOT** remain in the recogniser.

Parameter	Governs	Provisional
<code>pull.velocity_min</code>	speed toward M	0.25 m/s
<code>pull.step_divisor</code>	speed to step count	0.15
<code>pull.cooldown</code>	minimum interval	0.08 s
<code>zip.distance</code>	ribbons-together	0.10 m
<code>zip.closing_window</code>	tempo measurement window	0.30 s
<code>zip.frontier_max</code>	wave budget, in notches	<i>from reference device</i>
<code>unzip.distance</code>	ribbons-apart	0.22 m
<code>twist.height_delta</code>	vertical offset	0.06 m
<code>twist.hold_frames</code>	frames before firing	8
<code>halt.roll_fire</code>	head roll, firing angle	15°
<code>halt.roll_rearm</code>	head roll, re-arm angle	6°
<code>halt.hold</code>	hold before firing	0.25 s
<code>scissors.spread</code>	index/middle angle	0.40 rad
<code>scissors.extended</code>	extension floor	<i>to be derived</i>
<code>scissors.curled</code>	curl ceiling	<i>to be derived</i>
<code>loop.min_radius</code>	circle size floor	<i>to be derived</i>
<code>loop.radius_variance</code>	circularity tolerance	<i>to be derived</i>
<code>loop.plane_residual</code>	planarity tolerance	<i>to be derived</i>

6.13 Finger curl must be scale-invariant

The implemented curl metric is $\| \text{tip} - \text{metacarpal} \| / 0.08$ clamped to unity, with “curled” asserted below 0.30. A fully curled adult ring finger measures roughly 0.06–0.08 m tip-to-metacarpal, giving 0.75–1.0. The threshold is unreachable and scissors never fires. The metric is also not scale-invariant: the fixed divisor makes detection depend on hand size.

Curl MUST instead be the interior angle at the proximal interphalangeal joint, or tip-to-wrist distance normalised by that finger’s own summed segment lengths. Thresholds MUST be derived from recorded traces (§13), not chosen by inspection.

7 Snip semantics and the tune term

7.1 The defect being corrected

The engine currently derives a snip from the *left* cursor alone: it computes $from = p_L - w$ and $to = p_L$ for ribbon width w , then advances both fresh spigots to $from$. Since p_L and p_R are independent by design, the stored snippet is $(d(c_L, b_L, k), d(c_R, b_R, k))_k$ over a single index, while what M heard was $(d(c_L, b_L, i), d(c_R, b_R, j))$ with i and j independent.

A snipped tune is, in general, not the tune M heard.

A third representation compounds it: the tray receives the digits actually displayed, while the snippet map receives the re-derived pairs. “The snipped tune” denotes three different things depending on where one looks.

7.2 Resolution

$\text{Snip}(k, i_L, i_R, n)$ denoting $(d(c_L, b_L, i_L + t), d(c_R, b_R, i_R + t))_{t \in [0, n)}$.

The mesh guarantees equal counts on both sides, so a common length n is exact rather than assumed. The near hand supplies i_L and i_R ; the hand separation supplies n . Tray, snippet map and realised audio MUST all derive from this one term.

7.3 Consequences

The Theory of skeins. `Skein.module` currently declares

```
Snip . k:Skein, i:Nat, j:Nat |- "snip" "(" k " ", " i ", " j ")" : Tune;
```

inheriting the single-interval assumption. It becomes

```
Snip . k:Skein, iL:Nat, iR:Nat, n:Nat
  |- "snip" "(" k " ", " iL ", " iR ", " n ")" : Tune;
```

with the empty-snip equation restated as $(\text{Snip } k \ i_L \ i_R \ \text{NZero}) == (\text{Rest})$. A cut placed with the hands together gives $n = 0$ and normalises to `Rest` by that equation.

Twist commutes with snipping, for free.

Proposition 2. *Normalising $\text{Twist}(\text{Wv } l \ r)$ to $\text{Wv } r \ l$ also reinterprets i_L as indexing r , which is what the engine does—twist swaps whole spigots, configuration and cursor together. No additional equation is required.*

A run yields a family of tunes. Because the cut is virtual the band is never disturbed, so M may capture any windows she likes over one mesh: $\text{Snip}(k, i_L + a, i_R + a, m)$ for any offset a and length m inside the meshed span. They need not be adjacent, need not be disjoint, and may repeat. The relation between a performance and the tunes taken from it is readable off the syntax rather than stored in a link table—the same argument that made tunes terms in the first place—and overlap is visible in the syntax as shared intervals.

7.4 Interpretation lives in the envelope

Since the pitch and duration maps are changeable during a session (§3.3), the term alone is underdetermined: the same $\text{Snip}(k, i_L, i_R, n)$ sounds different under a different scale. The split is clean and **MUST** be observed:

The term	The envelope
which digits, paired how $\text{Snip}(k, i_L, i_R, n)$	pitch map, duration map, tempo, instrument copied in at capture time

The term is the material; the envelope is the interpretation. Breeding therefore operates on material and is map-agnostic, which is what is wanted when crossing two tunes made in different scales.

8 Performance recording

Definition 1 (Performance trace). An ordered sequence of timestamped records: an opening declaration of both stream configurations, the maps, tempo and instrument; then every gesture event with its parameters; then a closing timestamp.

The engine **MUST** record a trace for every session without being asked. The streams are deterministic and the state machine is total, so the trace **MUST** replay the session exactly; no audio is stored. Snips **MUST** appear in the trace, so a performance resolves to the tunes harvested from it and each tune resolves back to its performance. Timestamps **MUST** be monotonic at millisecond resolution or better, and records **SHOULD** carry the raw hand-tracking metrics that produced each gesture, for §13.

9 Wire protocol, version 3

Messages **MUST** be newline-delimited JSON, one object per line, each carrying **type** and **"v": 3**. Both sides **MUST** serialise via **Codable** and **serde**; neither side **MAY** construct JSON by string interpolation. The current client escapes only the double quote, so a name containing a backslash produces malformed JSON that the engine silently discards.

9.1 Client to engine

Type	Fields	Note
<code>pull_left</code> , <code>pull_right</code>	<code>steps</code> , <code>velocity</code>	
<code>twist</code>	—	rejected unless unzipped
<code>zip</code>	<code>closing_speed</code>	sets the mesh tempo
<code>unzip</code>	—	
<code>halt</code>	<code>on</code>	from roll direction; not a toggle
<code>mount</code>	—	stops the wave; enters meta
<code>unmount</code>	—	returns to play; mesh survives
<code>loop</code>	<code>on</code>	from rotation direction
<code>scissors</code>	<code>near</code> , <code>far</code>	two notch offsets; no name
<code>rename</code>	<code>id</code> , <code>name</code>	tray operation
<code>configure</code>	<code>left</code> , <code>right</code> , <code>pitch_map</code> , <code>duration_map</code> , <code>tempo</code> , <code>instrument</code>	
<code>quit</code>	—	

Three substantive changes. **scissors** carries two notch offsets, where the gesture currently fires a locationless event and the engine derives the window from the ribbon width—the assumption revision 3 removed. **zip** carries the closing speed, which is the tempo. **halt** and **loop** carry a direction-derived boolean rather than toggling, so a missed or doubled fire cannot invert the state.

9.2 Engine to client

Type	Fields
<code>digits</code>	<code>left</code> , <code>right</code> , <code>left_pos</code> , <code>right_pos</code>
<code>note</code>	<code>pitch</code> , <code>duration</code> , <code>velocity</code> , <code>left_pos</code> , <code>right_pos</code> , <code>notch_index</code>
<code>zip_state</code>	<code>zipped</code> , <code>front</code> , <code>running</code> , <code>tempo</code> , <code>budget_left</code> , <code>stopped_by</code>
<code>snip_ack</code>	<code>id</code> , <code>name</code> , <code>count</code> , <code>i_left</code> , <code>i_right</code>
<code>state</code>	<code>mode</code> , <code>looping</code> , <code>left_label</code> , <code>right_label</code> , <code>pitch_map</code> , <code>duration_map</code> , <code>instrument</code>
<code>status</code>	<code>text</code>
<code>error</code>	<code>code</code> , <code>text</code>

stopped_by **MUST** distinguish **halt**, **mount** and **exhausted**. The wave can now end without a gesture, and a run that stops mid-phrase with no explanation reads as a crash; the client **MUST** also give a visible approach signal as **budget_left** runs down. Playback state **MUST** be a field, never inferred from prose. The client currently clears its playing flag on **statusText.contains("stopped")**. Nothing in the client **MAY** parse **status**, which is advisory and human-readable only.

9.3 Rate discipline

The engine currently emits **status** on every frame of its 60 Hz loop, beneath a comment reading “Push status if changed” describing a check the code does not perform. Each message invalidates the entire client panel.

- **status**, **state** and **zip.state** MUST be emitted only on change.
- **digits** MUST be emitted only when a cursor moves, coalesced to at most 30 Hz.
- **note** MUST be emitted per note and MUST NOT be coalesced.
- Aggregate steady-state traffic SHOULD stay below 200 messages per second.

10 Visual model

Axis correction. `RibbonView.swift` places both ribbons at a shared `RIBBON_X` and separates them by `LEFT_Y`/`RIGHT_Y`—stacked vertically at the same left-right position. Under §4 they separate on x , left spool to the left hand. This MUST be corrected before any geometry is tuned.

Patches. Pooled and reused. Text meshes and materials MUST be cached per digit value and per depth bucket and MUST NOT be regenerated per frame; `PatchEntity.configure` currently calls `MeshResource.generateText` and builds a fresh physically-based material for every patch on every update, which is the dominant cost in the render path.

The three sections MUST be visually distinct: future occluded on the spool, present taut and legible, past hanging below the hands and dimmed.

The mesh MUST be rendered at the notch scale. The zip front MUST be a distinct, clearly visible entity, since it is the playhead M watches. It no longer needs to be *targetable*: halt is a head gesture, so nothing aims at the front, and any minimum apparent size is an aesthetic choice rather than a functional requirement.

Far-field compression. As the wave runs, the spools MUST compress toward point sources well before the frontier budget is exhausted, so that the cost of a long play is visible in z -axis real estate. The compression MUST be continuous; a discontinuity will read as a glitch.

The virtual cut MUST be rendered as a copy lifting clear of an intact band. Showing a gap, a severed end or a falling stub would misrepresent the mechanism and lead M to expect material to be consumed.

Hand ghosts MUST mirror the tracked joints. The recogniser receives every hand anchor and discards it; the ghost entity is added to the scene and never updated.

The tray MUST display the digits actually snipped, not the live ribbon contents.

11 Audio model

Audio MUST be synthesised on the headset. In the current architecture the engine drives CoreMIDI on the Mac, so M pulls a ribbon in front of her face and hears the result from a laptop across the room, after a network hop and a MIDI buffer. That makes the latency

figure—the most important measurement of the September sessions—unrepresentative of any shipping configuration.

- The client **MUST** render **note** messages through an on-device sampler.
- Audio **SHOULD** be spatialised so each ribbon sounds from its own position. The two streams are distinct objects in front of M and should be audibly distinct.
- Mac-side MIDI **MAY** be retained for recording to a workstation and **MUST NOT** be the default.

12 Non-functional requirements

Property	Target	Maximum
Notch engagement to audible note	30 ms	50 ms
Head roll to front freezing	one frame	two frames
Gesture to visual response	one frame	two frames
Rendered frame rate	90 fps sustained	—
Digit emission, memoised range	—	1 ms per 1000 digits
Engine to client message rate	$<100\text{ s}^{-1}$	200 s^{-1}
Session length without degradation	45 min	—

Failure modes **MUST** be visible and recoverable. Channel loss **MUST** surface in the panel and retry with backoff, preserving engine state. Denial of hand-tracking authorization **MUST** produce an explanatory message and fall back to panel controls, not a console line. The tray **MUST NOT** silently discard entries; it currently caps at twenty and drops the oldest.

13 Instrumentation

The thresholds are guesses, one is provably unreachable, and there is no data to tune against.

- A debug overlay **MUST** display live metrics per hand: separation, palm velocity, height differential, per-finger extension and curl, accumulated rotation angle, and which predicates are satisfied.
- Raw joint transforms **MUST** be recordable at tracking rate with a manual label marker.
- Thresholds **MUST** be tunable offline and reloadable without a rebuild.

The first half-day of headset time **SHOULD** capture a labelled corpus rather than play. Those traces are the same artifact as a performance trace, so the work is a first delivery of the product rather than a detour from it.

14 Publishing and consent

Everything the instrument produces — tunes, performance traces, gesture corpora — is M’s, and her back-end data is guarded by her private key. Recording every session by default is therefore safe: nothing becomes visible to anyone else without an explicit publishing action taken by M.

- The engine **MUST** record every session’s performance trace without asking.
- No artifact **MAY** leave the device, or become visible to any other party, except through an explicit publish action.

- Publishing **MUST** be a distinct, deliberate act, never a side effect of capture, naming, or any gesture in §6.

15 Deferred

Recorded here so the reasoning is not lost.

Rests in captures. A halt places a silence that is preserved in the performance trace but absent from any tune captured across it. Carrying the silence would be expressible with machinery the theory already has — since a mesh has one tempo, a wall-clock halt quantises to a definite number of rests, and the capture becomes $\text{Seq}[\text{Snip}(k, i_L, i_R, a), \text{Rest}^m, \text{Snip}(k, i_L+a, i_R+a, b)]$ with no new constructors. It is deferred because it makes a capture no longer always a single leaf, and tunes with differing internal structure are harder to cross than flat runs. Revisit after the breeding engine is exercised.

Gestural map control. Changing pitch and duration maps by gesture rather than by text. Deferred to keep the vocabulary at ten.

16 Conformance checklist

1. Clean install on device; entitlements and provisioning verified.
2. Hand-tracking authorization requested, granted, denial handled visibly.
3. Device-pose authorization confirmed and granted.
4. Engine discovered automatically; connection indicated within five seconds.
5. All ten gestures fire, each confirmed in the debug overlay, before any music is attempted.
6. Hand ghosts visible and tracking.
7. Zip propagates unaided; each notch pair sounds from the headset; spools compress as it runs.
8. Closing the hands faster yields an audibly faster run.
9. Head roll one way freezes the front, the other resumes, and repeats in the same direction are no-ops.
10. Tempo is unchanged across a halt and resume.
11. The wave stops of its own accord at the frontier, with an approach signal beforehand and **stopped_by** reporting **exhausted**.
12. Mount stops the wave and enters meta; unmount returns to play with the mesh intact.
13. Pull, twist, zip and unzip do not fire in meta; loop and scissors do not fire in play.
14. Loop set and cleared by rotation direction; loop span runs mount to front.
15. Snip with two hands yields a tune of exactly the bracketed notch count, auto-named.
16. The band is visibly intact after a snip, and a second overlapping snip succeeds.
17. A captured tune, re-realised from (k, i_L, i_R, n) under its envelope, is identical to what was heard.
18. A capture spanning a halt contains no silence; the performance trace replays the silence.
19. 90 fps sustained with sixty patches per ribbon.

20. Tray and performance trace survive an application restart.
21. Gesture traces written to disk.

Items 17 and 18 are the acceptance tests for §7; neither can pass on the current build.

17 Triage against the demo deadline

Not all of this is needed to demonstrate the instrument. Ranked by what a demo actually requires:

Blocking — without these there is no demo. Transport off the Unix socket (§2); on-device audio (§13); the scale-invariant curl metric, without which scissors cannot fire at all; the four unwired visual features; and the axis correction, since the ribbons are currently stacked on *y*.

Core to the idea. Zip with propagation and tempo from closing; halt by head roll; mount, unmount and the mode split; the two-handed virtual snip.

Deferrable if time runs short. Loop, which needs the trajectory detector — the demo survives without it, though auditioning before capture is a large part of what makes the instrument feel like an instrument. Far-field compression, which is expressive rather than functional. Textual map control, if a single sensible default scale is chosen. The frontier bound, if the demo runs are short.

Not deferrable despite looking like it. The gesture-trace recorder. Every threshold in §6.12 is a guess, and without recorded traces the tuning after the first session will be guesswork too.

18 Decisions sought

The ten decisions raised by revision 3 are settled and incorporated. What remains:

1. **The frontier figure.** `zip.frontier_max` is specified as “a generous looper phrase”. Set the number from a reference device.
2. **Roll thresholds.** 15° firing and 6° re-arm are provisional and should be confirmed against recorded traces — players lean into phrases, and the false-fire rate is unknown.
3. **Auto-naming scheme.** Take number, or a name derived from the address itself. The latter is more informative and less legible.
4. **Default maps.** Which pitch and duration map, over which root, when a session opens.
5. **Configuration ordering.** S1 tethered for the demo with S2 to follow, or straight to S2. About three days against about a week, with the demo deadline the deciding factor.